

NHL Game Outcomes

Hayden Bradley

ORB 648: Machine Learning

Diksha Godbole

31 May 2025

Introduction:

In the world of sports, teams in every professional sports environment use data analysis to give their teams a predictive edge over others. Hockey is no different; in a fast-paced sport, people use various factors to try to predict whether a team will win a hockey game. This project delves into the world of hockey, exploring how machine learning can be applied to predict the outcomes of a hockey game. It will cover the entire process, from data acquisition and cleaning to data splitting, training, testing, and visualizing the results.

Data:

I first selected my data from the Kaggle website, using the NHL-Dataset, which I found contained many files. I selected three main files to use in my project `game.csv`, `team_info.csv`, and `game_teams_stats.csv`. Game (`game.csv`) primarily consisted of the `game_id`, `season`, `team_ids`, and the number of goals each team scored. Team info (`team_info.csv`) contained the main information on the teams themselves, including the team ID and the team name. Lastly, game teams (`game_teams_stats.csv`) had most of the information that made this project possible, it contained the `season_id`, `game_id`, `team_id`, `won` (whether the team won or not), `goals`, `shots`, `hits`, `pim` (penalties in minutes), `power_play_opportunities`, `power_play_goals`, `faceoff_win_percentage`, `giveaways`, `takeaways` and `blocked` (blocked shots).

To create one dataset, I then had to merge the three datasets into one. I achieved this by first merging the `game` and `game_teams` datasets by `game_id`, which both datasets shared. This provided me with the new merged data set, which I then named `game_merged_data`. I then merged the `team_info` file with the previously merged `game_merged_data` by joining them on the `team_id`.

After merging all the values, I then had to clean the data. I first checked if there were any null values in my dataset. I originally had 62,890 rows of data, but I then dropped all rows that contained any null values. This dropped the number of rows to 39,446. In the future, instead of dropping all values that had nulls, I should have dropped all rows that didn't contain any of the variables I was actively using in my project. So if hits had a null, it would drop since I am actively using it in my project. But if the venue location had a null value, I shouldn't have dropped it because it wouldn't affect my calculations in the future.

Next, it came time for me to split the dataset. To create my X, I had to find which columns would be beneficial to calculating my results. I concluded to only add the following columns to my calculations, 'shots', 'hits', 'pim', 'powerPlayOpportunities', 'faceOffWinPercentage', 'giveaways', 'takeaways', 'blocked'. The reason for not adding any other column was mainly that most columns just contained information about the game, which would not be important to the calculations, such as the time zone in which the game was played. The only numerical values that I did not use were goals, home goals, and away goals. I did not use these values because when it comes to scoring, it would have a direct correlation as to whether a team wins or loses. If you score more than your opponent, you win the game. The target variable for prediction was the column called 'Won'. This value was a binary value that told you whether a team won or not.

Methodology:

To get my results to predict if a team wins a game or not, I used two different classification algorithms, namely, random forest and decision tree. I prepared the data set by creating an 80-20 split. I used the 80% as my training set and the remaining 20% as my testing set. To determine the overall effectiveness of my model, I used the accuracy and confusion matrix. The accuracy is used just to see the overall percentage of the time it correctly predicted

the actual outcome. The confusion matrix shows the number of true positives (correctly predicted wins), false positives (incorrectly predicted wins), true negatives (correctly predicted losses), and false negatives (incorrectly predicted losses).

I also used a feature importance model to see, after my initial testing, which variables contributed the most to the models themselves. This outcome helped me identify which statistics are the most important for predicting the correct result of a game. My feature importance showed me that the most important features are face-off win percentage, shots, hits, and giveaways. I was surprised by this result, and initially, before seeing these results, I believed that shots would have been the most influential to determining if a team wins or not since the more shots you take, the more likely you are to score and the more you score, the more likely you are to win the game. After the initial feature importance outcome, I created another set of data that I only used for my X, 'shots', 'hits', 'pim', 'faceOffWinPercentage', 'giveaways', 'takeaways', and 'blocked', I named this set X1.

Results:

My initial results from my decision tree model accuracy came back at 0.7636, and random forest came back at 0.7892, showing that my random forest model was more accurate. The confusion matrix for the decision tree was $\begin{bmatrix} 2991 & 916 \\ 949 & 3034 \end{bmatrix}$, and the random forest confusion matrix was $\begin{bmatrix} 3116 & 791 \\ 872 & 3111 \end{bmatrix}$. This meant that my random forest model accuracy was .0256 more accurate than my decision tree model accuracy. The random forest model was also able to predict 125 more true negatives and 125 fewer false negatives than the decision tree model, all while predicting 77 fewer false positives and 77 more true positives.

I then used the feature importance model to make multiple predictions using different variables to see if I was able to get a higher accuracy score compared to my first model. Only

one of my later models, X1, was able to get a higher accuracy score. This model used 'shots', 'hits', 'pim', 'faceOffWinPercentage', 'giveaways', 'takeaways', and 'blocked'. This model achieved a random forest model accuracy score of 0.7909 and a confusion matrix of $\begin{bmatrix} 3116 & 791 \\ 859 & 3124 \end{bmatrix}$. Making this model 0.0017 more accurate than my original model and predicting the same number of true and false negatives, I was able to predict 13 fewer false positives and 13 more true positives.

Conclusion:

In conclusion, this project successfully demonstrated the application of machine learning techniques to predict whether a team wins a game using in-game team statistics. The features highlighted in my feature importance model revealed key statistical characteristics that determine whether a team wins or loses. This project shows the growing field of sports analytics and how machine learning can be used to predict games and help teams create an advantage over their opponents. Future models I can use include player statistics, team trends over a given month, trends a team has against a certain team, whether home or away has a factor, and even the amount of travel and the number of days of rest between games. Overall, this project significantly enhanced my understanding of the importance of machine learning and its applications in various environments.